Orchestrator Experiment Report

Runs: T34 Phase Transition and P3 Learning From Failure

Date: 2026-05-04

Project: LlmFromScratch — dynamic-flow orchestration experiments

---

Document Purpose

This report describes in plain language what happened across two experiment runs completed on 2026-05-04: the T34 Phase 3 to Phase 4 Transition Gate (code phase 24) and the P3 Learning From Transition Failure (code phase 25).

For each run it explains:

- what problem the run was trying to solve,
- what oracle documents were produced before the code was written,
- what code and diagrams were produced or changed,
- what the test results showed,
- and what the results mean for the broader architecture.

The architecture change section describes the four UML diagrams before and after the corrections made in T34 Round B, with specific reference to every entity, interface, and parameter that was added or changed.

---

1. Background: What the Orchestrator Is and Why Phase Transitions Matter

1.1 The Nine-Node Pipeline

The LlmFromScratch project implements a dynamic-flow orchestration pipeline in pure C# without any external ML frameworks. The pipeline is a nine-node sequence that processes one source packet — a structured problem description — into a human-readable result.

The nine nodes are:

Node	Role
Node 1	Domain Splitter: breaks the source packet into domain-tagged fragments
Node 2	Domain Translator: converts each fragment into a formal domain statement
Node 3	Scheme Planner: produces a step-by-step plan for the domain statement
Node 4	Domain Processor: executes each plan step
Node 5	Applier: applies the processed plan to produce concrete results
Node 6	Result Translator: converts formal results into natural language
Node 7	Arbiter Selector: chooses which review units should fire
Node 8	Arbiter Units: review results and produce accept/concern/reject verdicts
Node 9	Formatter: assembles the final human-readable output

The critical technical foundation is that the system does not process words directly. Every input passes through a semantic compiler that maps text to a 22-slot predicate vector. The 22 predicates are a fixed vocabulary:

```
NULL_GUARD   FAIL        ITERATE     HANDLE_ERROR  AWAIT
THROW        DECLARE     ASSIGN      CREATE        CALL
BRANCH       RETURN      GET_OP      VALIDATE_OP   CREATE_OP
UPDATE_OP    DELETE_OP   PROCESS_OP  CONVERT_OP    SEND_OP
LOG_OP       AUTH_OP
```

These predicates are the fixed contract between all units in the system. Domain labels (such as database-query-efficiency or api-contract) are attached to predicate vectors as metadata. Phase signatures (such as Phase 3 — Technical Design) are learned statistical patterns over predicate vectors, not fixed code structures.

1.2 Why Phase Transitions Are Needed

As of the start of the T34 experiment, the nine-node pipeline could process one source packet in isolation. It did not know that its output would become the input for a downstream phase of

development work. It did not carry decisions, constraints, or knowledge gaps forward from one phase to the next.

A real development lifecycle has seven phases: UI/UX Definition, Functional Specification, Technical Design, Implementation, Testing, QA, and QA Automation. Each phase produces decisions that the next phase must respect. If the Technical Design phase decides that all services must inherit from BaseService, the Implementation phase must know about that decision and either enforce it or explicitly name it as an uncovered gap.

The experiments described in this report build and prove the mechanisms that allow the system to carry phase context forward and verify that downstream phases are consistent with what upstream phases decided.

Three structures need to be added to the architecture for this to work. They sit above and beside the nine-node pipeline:

1. The **Phase Context Packet** — a structured artifact that every completed phase produces, capturing its decisions, inherited constraints, knowledge edges (named gaps), boundary markers, and rejected items.
2. The **Transition Gate** — a verification unit that runs between two phases and checks whether the downstream phase's source packet is consistent with the upstream phase's context packet.
3. The **Phase Registry** — a versioned topology that maps phase nodes, transition edges, continuity thresholds, and conformance tests. This is stored as data, not code, so that methodology changes do not require rebuilding the system.

---

2. T34 Experiment — Phase 3 to Phase 4 Transition Gate

2.1 What T34 Was Trying to Prove

T34 (code phase 24, experiment identifier T34-PHASE-3-TO-4-TRANSITION-GATE) was the first concrete test of the transition gate concept. It asked one specific question: given a real Phase 3 Technical Design context packet and a real Phase 4 Implementation source packet, does the transition gate correctly identify what was preserved, what was left as a named gap, and what was safe to pass despite appearing superficially problematic?

The Phase 3 source was the existing XIIGEN dynamic-flow architecture documents: - `DYNAMIC-FLOW-ARCHITECTURE-PLAN-v1_0-DRAFT.md` - `DYNAMIC-FLOW-HIGH-LEVEL-DESIGN-AMENDMENT-v1.md` -

`DECISIONS-LOCKED.md`

The Phase 4 source was an existing implementation plan: - `AUTONOMOUS-LEARNING-SESSION-PLAN-v1.0.md`

The implementation plan described an eight-step autonomous learning cycle for the `16.AutonomousLearning` project component.

Three specific scenarios were designed to test the gate's judgment:

Scenario T34-1: Stated-but-unenforced constraint. The Phase 3 design requires that all production services inherit from `BaseService`. The Phase 4 plan mentions BaseService in its dependency map but does not include a Gate B check or test assertion that would catch a service that omits the inheritance. Expected verdict: CONCERN (not BLOCK — the phase can proceed with this named gap, but the gap must be resolved before the phase output is accepted).

Scenario T34-2: Absent knowledge-edge acknowledgment. The Phase 3 design introduces a cross-phase context packet mechanism for carrying decisions between phases. This mechanism is not yet implemented. The Phase 4 plan does not mention this gap explicitly — it neither implements it nor marks it as out of scope. Expected verdict: CONCERN (the local component can proceed, but the plan must explicitly mark the gap as out of scope so it does not silently disappear).

Scenario T34-3: Superficial resemblance to a rejected item. The Phase 4 plan lists a fixed five-step implementation sequence (Phase 1 through Phase 5 build steps). The Phase 3 design explicitly rejected a static runtime pipeline as an architectural shape. A naive checker would flag the fixed sequence as reinstating the rejected item. Expected verdict: PASS — the fixed sequence is build-time implementation order, not runtime execution topology. Artifact types are different.

2.2 T34 Round A — Oracle Documents

Round A produced five oracle documents that defined the experiment before any code was written. These documents are the authority; the code must match them, not the other way around.

T34-SOURCE-BINDING-v1.md defined the upstream and downstream source documents and produced a complete upstream context packet. The context packet contained:

- 6 decisions made (DCTX-01 through DCTX-06), including the locked decisions that the pipeline graph is data not code, that internal LLMs run before any external AI, and that arbiters are the hard acceptance boundary.

- 7 constraints inherited (CSTR-01 through CSTR-07), including no external ML frameworks, StackDetector before normalization, StratifiedSplit for splits, the D-09 weight formula, and BaseService-style service inheritance.
- 3 knowledge edges (KEDGE-01 through KEDGE-03), including the cross-phase context packet mechanism not yet implemented and the swappability judge not yet trained.
- 2 boundary markers (BND-01, BND-02).
- 4 rejected items (REJ-01 through REJ-04), including the static runtime pipeline as an architecture shape and external AI as ordinary control-path authority.

T34-TRANSITION-DOMAIN-MAP-v1.md defined the formal vocabulary for the four transition domains: constraint preservation, decision acknowledgment, knowledge edge continuity, and rejection preservation. For each domain it specified the formal statement shape, the key vocabulary, and the required phrasing for each of the three T34 scenarios.

T34-TRANSITION-FAILURE-TAXONOMY-v1.md defined six failure types (F1 through F6) with their observable signals and correct verdicts. The taxonomy established the critical distinction: F2 (stated but unenforced) produces CONCERN, not BLOCK. F3 (absent knowledge edge acknowledgment) produces CONCERN. F6 (superficial reinstatement with different artifact type) produces PASS.

T34-TRANSITION-HUMAN-RESULT-STANDARDS-v1.md defined what a technical lead must be able to read in the final human result: the transition verdict must be PASS WITH GAPS, five preserved constraints must be named, each concern must have a concrete remediation, and the build-time sequence must be explicitly distinguished from the rejected static runtime pipeline.

T34-TRANSITION-EXECUTOR-CONTRACT-v1.md specified the seven pass gates, the required metrics table, the required test names, and the required evidence output (JSON, Markdown, and Mermaid files).

2.3 T34 Round B — Code Changes

Round B implemented the deterministic transition proof as a C# class in the `35.ExperimentsV2` project.

New file: `35.ExperimentsV2/Experiments/T34PhaseTransitionProof.cs`

The class `T34PhaseTransitionProof` is in namespace `ExperimentsV2`. It contains twelve record types and fourteen methods. The key structures are:

Record: PhaseContextPacket Parameters: `PacketId`, `UpstreamPhase`, `DownstreamPhase`, `DecisionsMade[]`, `ConstraintsInherited[]`, `KnowledgeEdges[]`, `BoundaryMarkers[]`,

`RejectedItems[]`. Purpose: carries the complete upstream phase authority that the transition gate checks against the downstream packet set.

Record: DomainPacket Parameters: `PacketId`, `DomainLabel`, `CoverageStatus`, `PromptFragment`, `ReasonForInclusion`, `SourceIds[]`. CoverageStatus values: `covered`, `concern`, `boundary`. Purpose: one fragment produced by the transition domain splitter (node-01 equivalent), carrying one upstream item into the translation layer.

Record: FormalTranslation Parameters: `StatementId`, `SourcePacketId`, `CoverageStatus`, `HandoffMarker`, `FormalStatement`. HandoffMarker values: `ready_for_transition_planner`, `send_to_transition_gate_with_concern_flag`, `send_to_transition_gate_with_scope_boundary`. Purpose: the output of node-02 translation for one domain packet.

Record: TransitionScenario Parameters: `ScenarioId`, `Title`, `Verdict`, `VerdictType`, `Reason`, `EvidenceIds[]`, `RequiredPreservations[]`, `Blocks[]`. VerdictType values: `pass`, `concern`, `block`. Purpose: one planned or applied transition check from node-03 through node-05.

Record: ArbiterVerdict Parameters: `Arbiter`, `TargetScenarioId`, `Verdict`, `VerdictType`, `BlocksTransition`, `Reason`. Purpose: the node-08 arbiter classification for one scenario.

Record: Metrics Parameters: `LostConstraintCount`, `DecisionAcknowledgmentMissingCount`, `KnowledgeEdgeDroppedCount`, `RejectionLostCount`, `ConcernCount`, `BlockCount`, `PassCount`, `DomainPacketCount`, `FormalTranslationCount`, `ScenarioCount`, `ForbiddenRuntimeTopologyReinstatementCount`, `HumanResultSectionsPresent`, `TransitionSuccess` (JSON key: `transition_success`). Purpose: the machine-readable gate collection that determines whether all pass gates pass.

Record: Evidence This is the root record that `WriteEvidence()` serializes to JSON. It contains: `ExperimentId`, `ScenarioId`, `RunId`, `RunTimestampUtc`, `SourceBindingPath`, `ExpectedTracePath`, `SourceBinding`, `Node00CompiledPhaseContext`, `Node01DomainPackets[]`, `Node02FormalTranslations[]`, `Node03ScenarioPlan[]`, `Node04ScenarioProcessing[]`, `Node05AppliedTransitionResults[]`, `NodeTrace[]`, `EdgeTrace[]`, `Node08ArbiterVerdicts[]`, `Node09HumanSections[]`, `FinalHumanResult`, `Metrics`, `PassGates[]`, `OverallVerdict`, `Summary`.

The build pipeline inside BuildEvidence():

```
BuildSourceBinding()
  → BuildPhaseContextPacket()
    → BuildDomainPackets(phaseContext)          [10 packets]
      → BuildFormalTranslations(packets)         [10 translations]
        → BuildScenarioPlan(translations)        [3 scenarios]
          → BuildScenarioProcessing(plan)
            → BuildAppliedTransitionResults()
              → BuildArbiterVerdicts(applied)
```

```
                → BuildHumanSections(verdicts)
                  → BuildFinalHumanResult(sections)
  → BuildNodeTrace(...)
  → BuildEdgeTrace(...)
  → BuildMetrics(...)
  → BuildPassGates(...)
```

The `Mentions()` method is the core predicate-matching function. It maps each upstream constraint or decision to the domain packet that carries it, using keyword matching against the 22-predicate vocabulary labels. For example, `sourceItem.Contains("BaseService")` returns `true` only for `td06_base_service_constraint`.

Ten domain packets produced by BuildDomainPackets():

Packet ID	Domain label	Coverage status
td01_runtime_orchestration_decision	runtime_orchestration_decision	covered
td02_internal_first_authority	internal_first_authority	covered
td03_arbiter_authority	arbiter_authority	covered
td04_idle_learning_promotion	idle_learning_promotion	covered
td05_downstream_phase_sequence	implementation_sequence_artifact	covered
td06_base_service_constraint	implementation_constraint_preservation	concern
td07_cross_phase_context_packet	cross_phase_context_handoff	concern
td08_static_runtime_rejection	rejected_runtime_shape	covered
td09_downstream_scope_boundary	downstream_scope_boundary	boundary
td10_transition_verdict_assembly	transition_verdict_assembly	covered

New file: `Tests/DynamicFlowT34Tests.cs`

Eight tests in class `DynamicFlowT34Tests`:

Test name	What it verifies
T34_Run_WritesJsonMarkdownAndMermaidEvidence	JSON, MD, MMD files are created with matching run IDs
T34_EvidenceJson_UsesRequiredMetricKeys	`transition_success` key exists in JSON; no misspelling (`transition_sucess` must not appear)
T34_Run_PassesPhase3ToPhase4TransitionGates	OverallVerdict = PASS_WITH_GAPS, all 10 pass gates pass
T34_Node01_EmitsExpectedTenDomainPackets	All 10 packet IDs present; td06 and td07 have status concern
T34_BaseServiceConstraint_IsConcernNotBlock	T34-1 verdict = concern, blocks = false, human result names enforcement gap
T34_CrossPhaseContextEdge_IsConcernNotDropped	T34-2 verdict = concern, blocks = false, no dropped items in EdgeTrace
T34_BuildTimeSequence_DoesNotReinstateStaticRuntimePipeline	T34-3 verdict = pass; forbidden phrase not in human result
T34_HumanResult_IsPassWithGapsAndActionable	5/5 sections present; all required phrases present

2.4 T34 Round B — UML Diagram Changes

The T34 Round B manifest recorded that all four SVG diagrams were redrawn before the code was written, because the existing diagrams showed the wrong mental model. The corrections are explained below.

Diagram 1: `dynamic_flow_runtime_uml.svg`

What the diagram showed before correction:

The diagram represented the nine nodes as processing text or tokens directly. There was no semantic compiler at the front. There was no predicate packet as the fixed inter-unit contract. The diagram did not show a phase context compiler, a transition gate, a context packet store, or the continuous learning feedback path.

What the diagram shows after correction:

The diagram is organized into three horizontal lanes.

Lane 1 — Intake and Predicate Contract: - `User / Document Input` → `Semantic Compiler` → `Predicate Packet`

The Semantic Compiler maps all incoming text to the 22-slot predicate vector. The Predicate Packet is the fixed contract that every downstream node receives. No node sees raw words; every node sees predicate counts, domain tags, confidence scores, and dependency edges.

Lane 2 — Dynamic Flow Pipeline: The nine nodes appear in sequence: `Topology Router` → `Sub-Phase Router` → `Domain Splitter` → `Domain Translator` → `Scheme Planner` → `Processor and Applier Arrays` → `Convergence Gate` → `Result Translator + Formatter`.

Two new components appear after the pipeline: `Phase Context Compiler` and `Transition Gate Unit`. The Phase Context Compiler takes the complete predicate output of a phase run and produces a ranked set of predicate fragments (decisions, constraints, knowledge edges, boundaries, rejected items) for the next phase. The Transition Gate Unit receives the upstream carry-forward summary and the downstream source packet and produces PASS, PASS WITH GAPS, or BLOCK.

`Arbiter Selector + Units` and `Knowledge Edge Path` appear as the review layer that feeds the learning cycle.

Lane 3 — Registries and Learning: `Flow Registry`, `Unit Registry`, `Dataset Registry`, `Decision Graph`, `Context Packet Store`, `Event Log`, and `Continuous Learning`.

The Context Packet Store is a new registry that holds versioned phase predicate summaries. The dashed lines from the pipeline to the registries represent runtime lookups, not control flow.

Diagram 2: `dynamic_flow_data_layer_uml.svg`

What the diagram showed before correction:

The data layer diagram showed simpler structures that did not include phase topology, context packets, transition evidence, or learning gaps. The predicate vector was not represented as a concrete entity with named fields.

What the diagram shows after correction:

The diagram is an entity-relationship layout with eight top-row entities and four bottom-row entities.

Top-row entities:

`FlowRegistry.TopologyContract` — fields: `flowId`, `topologyKind` (request or phase), `nodes[]`, `edges[]`, `versionPointer`. This is how the nine-node pipeline topology is stored as data, not code.

`PhaseTopology` — fields: `methodologyId`, `phaseNodes[]`, `transitionEdges[]`, `semanticThresholds[]`, `companyVersion`. This entity is new. It stores the phase lifecycle as a versioned topology in the same registry family as flow topologies. A methodology change is a version pointer move, not a code deployment.

`UnitRegistry.Cell` — fields: `cellId`, `role`, `domainTagSet`, `predicateContract`, `trustScore`, `version`.

`RoleDomainSlot` — fields: `role`, `domainTagSet`, `activeCellId`, `fallbackCellIds[]`, `conformanceSuiteId`. This is the slot that a cell fills in the unit registry; a role-domain pair can have multiple fallback cells.

`PredicateVector` — fields: `predicateCounts[22]`, `domainTags[]`, `confidence`, `dependencyEdges[]`, `sourceEvidenceIds[]`. The array size 22 corresponds to the 22-predicate vocabulary. This entity is the fixed contract.

`DatasetRegistry.CorpusSlice` — fields: `sliceId`, `role`, `domainTagSet`, `learningPairs[]`, `sourceTrust`, `trainedCells[]`.

`DecisionGraph.RoutingEdge` — fields: `edgeId`, `fromSignature`, `toRoleDomainSlot`, `confidence`, `observations`. This entity encodes learned routing confidence as data.

`KnnMemoryEntry` — fields: `predicateVectorId`, `neighborId`, `arbiterStatus` (positive, warning, or rejected), `failureReason`.

Bottom-row entities (all new):

`ContextPacketStore` — fields: `contextPacketId`, `phaseRunId`, `predicateSummaryIds[]`, `transitionRole`, `version`, `sourceRun`. This entity stores the carry-forward predicate summaries between phases.

`TransitionEvidence` — fields: `transitionId`, `upstreamContextId`, `downstreamVectorId`, `similarityScores[]`, `verdict` (PASS, GAPS, or BLOCK). This entity stores the evidence that the transition gate produced for each phase-to-phase boundary.

`PhaseLearningGap` — fields: `gapId`, `phaseId`, `subPhaseId`, `missingPredicateVectorId`, `targetCellRole`, `feedbackSignal`. This entity tracks what a specific phase or sub-phase could not cover, so the learning cycle knows which cell to train.

`EventLog.EventRecord` — fields: `eventId`, `timestamp`, `sourceEntity`, `parentState`, `childState`, `trustScoreAtTime`. This entity is the append-only audit trail for all mutations.

Diagram 3: `continuous_learning_integration.svg`

What the diagram showed before correction:

The continuous learning diagram showed a general feedback loop without predicate-level specificity. It did not distinguish between real-time request feedback, idle scheduled learning, gap-driven cell creation, and phase-level learning.

What the diagram shows after correction:

The diagram shows four separate horizontal paths, each representing a distinct learning trigger:

Path 1 — Real-Time Request Feedback: `Pipeline Run` → `Arbiter Verdicts` (labeled: accept, concern, reprocess) → `Event Log`. Every pipeline run produces verdicts that are immediately appended to the event log. This path requires no idle window.

Path 2 — Idle Scheduled Learning: `Idle Detector` → `Feedback Router` (labeled: accepted, edited, rejected) → `Dataset Curator` → `LoRA Trainer` (labeled: incremental update) → `Promotion Gate` (labeled: after accuracy greater) → `Unit Registry`. The Idle Detector gates whether training is safe to run. The Feedback Router classifies stored event records. The Promotion Gate blocks adapter registration unless after-accuracy exceeds before-accuracy — this is D-14.

Path 3 — Gap-Driven Cell Creation: `Gap Detector` (labeled: sustained low confidence) → `Learning Gap Packet` (labeled: missing predicate evidence) → `Create or Fetch Cell` (labeled: trust-tier search) → `Unit Registry` (probationary cell) and `Decision Graph` (confidence updated). This path responds to sustained low confidence in a role-domain slot by creating a new probationary cell.

Path 4 — Phase-Level Learning: `Transition Evidence` → `Phase Learning Gap` → feedback into `Learning Gap Packet` (Path 3) and `Dataset Curator` (Path 2). This is the new path added by the T34 experiment design. Phase transitions produce transition evidence and phase learning gaps that feed into both the idle scheduled learning path and the gap-driven cell creation path.

Diagram 4: `missing_tools_for_context_completion.svg`

What the diagram showed before correction:

The diagram classified missing pieces as general methodology tooling — fixed schemas for phases, document templates, and checklist automation. This reflected a deterministic methodology mindset rather than a predicate-level adaptive design.

What the diagram shows after correction:

The diagram is reorganized into three sections based on implementation readiness, and all items are described as dynamic-flow cells over predicate vectors.

Section 1 — Exists and proven (4 items): Nine-node orchestration pipeline (proven by E53 deterministic proof); Semantic compiler contract (22 logical predicates + vectors); Arbiter authority (hard gate over KNN and external AI); Initial learning loop pieces (feedback, DPO, LoRA, idle gate).

Section 2 — Designed but not fully built (8 items): Flow and phase registry; Unit registry; Dataset registry; Decision graph; Intake and topology routers; Evidence pack selector; Budget and confidence gate; Context packet store.

Section 3 — Missing and now required (8 items): Phase context compiler cell; Transition gate cell; Phase learning gap tracker; Predicate-to-text translator; Swappability judge; Conformance runner; Marketplace trust tooling; Exam affordance UI.

The last section is the direct output of the T34 experiment design work. It names what must be built next, expressed as dynamic-flow cells rather than as fixed methodology features.

2.5 T34 Results

Overall verdict: PASS_WITH_GAPS **Transition success:** true **Domain packets:** 10/10 **Formal translations:** 10/10 **Transition scenarios:** 3/3 **Concerns:** 2 (T34-1 BaseService enforcement, T34-2 cross-phase context) **Blocks:** 0 **Passes:** 1 (T34-3 build-time sequence) **Human result sections:** 5/5 **Test baseline:** 997/997 → 1005/1005 (+8 tests) **Commit:** `e6622a8` pushed to main

The result confirmed three behaviors simultaneously:

1. The gate correctly identifies a named gap (BaseService unenforced) without blocking the phase.
2. The gate correctly identifies a knowledge edge that disappeared without acknowledgment, without blocking the phase.
3. The gate correctly distinguishes a build-time sequence from the rejected runtime topology, producing a clean PASS rather than a false BLOCK.

---

3. P3 Experiment — Learning From Transition Failure

3.1 What P3 Was Trying to Prove

P3 (code phase 25, experiment identifier P3-LEARNING-FROM-TRANSITION-FAILURE) extended T34 by asking: what happens when the transition gate does block, and does the system learn the right lesson?

P3 ran three sub-experiments in sequence:

P3-C — Corrupted run. The Phase 3 upstream context packet is unchanged. The downstream domain packet set is corrupted by removing the BaseService packet (`td06_base_service_constraint`) entirely. No packet in the set carries or acknowledges BaseService. Expected verdict: BLOCK.

This simulates a downstream implementation plan that is completely silent about BaseService — not mentioning it even as a named gap.

P3-L — Learning cycle. The BLOCK finding feeds into the idle learning cycle. The cycle produces one `FeedbackRecord` naming BaseService as the dropped predicate, then derives one `LearnedDiscriminator` that encodes the specific boundary: entirely absent → BLOCK; present as concern → PASS_WITH_GAPS.

P3-V — Verification run. The original T34 domain packet set (all 10 packets including td06 with status concern) is run through the transition gate. The gate consults the discriminator registry, finds the discriminator inactive because td06 is present, and produces PASS_WITH_GAPS — the same result as T34. The discriminator did not over-generalize.

The key question that P3 answered was: does the learned discriminator correctly distinguish between "entirely absent" and "present but unenforced"? These two states look superficially similar (both indicate the constraint is not fully enforced) but have very different meanings. An entirely absent predicate means the downstream plan has no awareness of the constraint at all — it cannot possibly enforce what it does not know about. A present-but- concern predicate means the plan knows about the constraint and has explicitly flagged the gap.

3.2 P3 Round A — Oracle Documents

Seven oracle documents were produced before the code was written.

P3-SOURCE-BINDING-v1.md defined the three sub-experiments, the shared upstream source (identical to T34), and the specific corruption applied in P3-C (removing td06 from the domain packet set). It also specified the six binding rules, including: the corruption is in the downstream domain packet set, not in the upstream authority; the P3-C BLOCK must name BaseService

specifically; and the P3-V verification must produce PASS_WITH_GAPS or the learning cycle has over-generalized.

P3-FAILURE-DISCRIMINATOR-v1.md defined the core distinction at the predicate level. It provided the exact evidence tables:

For T34 (PASS_WITH_GAPS case):

```
ConstraintsInherited: ["BaseService inheritance is a production service pattern..."]
DomainPackets: [{PacketId: "td06_base_service_constraint", CoverageStatus: "concern", ...}]
Mentions(packet, "BaseService") = true for td06
LostConstraintCount = 0
Verdict = PASS_WITH_GAPS
```

For P3-C (BLOCK case):

```
ConstraintsInherited: ["BaseService inheritance is a production service pattern..."]
DomainPackets: [9 packets, none referencing BaseService]
Mentions(packet, "BaseService") = false for all 9 packets
LostConstraintCount = 1
Verdict = BLOCK
```

P3-LEARNING-CYCLE-TAXONOMY-v1.md defined the FeedbackRecord shape, the LearnedDiscriminator shape, and the LearningCycleVerdict shape. It specified what the learning cycle must NOT produce: it must not create a general rule that blocks any constraint lacking downstream enforcement. The discriminator is BaseService-specific by design.

P3-EXPECTED-TRACE-v1.md provided the oracle for all three sub-experiments with the exact expected values for packet counts, verdict strings, metric names, and discriminator states.

P3-HUMAN-RESULT-STANDARDS-v1.md defined what human text must appear in each sub-experiment's result, including required sections and forbidden statements for each of P3-C, P3-L, and P3-V.

P3-EXECUTOR-CONTRACT-v1.md specified eight pass gates, the required metrics table (17 metrics), eight required test names, and the required evidence output structure.

3.3 P3 Round B — Code Changes

New file: `35.ExperimentsV2/Experiments/P3LearningFromFailureProof.cs`

The class `P3LearningFromFailureProof` is in namespace `ExperimentsV2`. It shares the upstream `PhaseContextPacket` and `DomainPacket` record shapes with T34, and adds five new record types:

Record: FeedbackRecord Parameters: `FeedbackId`, `SourceExperimentId`, `SubExperimentId`, `BlockedScenarioId`, `DroppedConstraint`, `UpstreamAuthority`, `BlockVerdict`, `LostConstraintCount`, `RejectedPattern`, `LearningNote`. Purpose: captures one BLOCK finding for the learning cycle. Each feedback record is keyed by the specific predicate that was dropped, not by a general failure category.

Record: LearnedDiscriminator Parameters: `DiscriminatorId`, `SourceFeedbackId`, `PredicateName`, `AbsentPattern`, `PresentButUnenforcedPattern`, `AbsentVerdict`, `PresentButUnenforcedVerdict`, `Specificity`, `Generalization`. Purpose: encodes the predicate-level boundary learned from a BLOCK finding. The `Generalization` field is always "none" for a correctly specific discriminator.

Record: LearningCycleResult Parameters: `CycleId`, `FeedbackRecordsAdded`, `DiscriminatorsAdded`, `DiscriminatorSummary`, `OverGeneralizationRisk`, `FeedbackRecords[]`, `LearnedDiscriminators[]`. The `OverGeneralizationRisk` field is the machine gate for specificity — it must equal "none" for the P3-L pass gates to pass.

Record: CorruptedRunResult Parameters: `SubExperimentId`, `CorruptedPacketCount`, `LostConstraintCount`, `DroppedConstraint`, `OverallVerdict`, `BlockCount`, `ConcernCount`, `PassCount`, `TransitionSuccess`, `Scenarios[]`, `HumanResult`. Purpose: carries the P3-C corrupted sub-experiment outcome.

Record: VerificationRunResult Parameters: `SubExperimentId`, `PacketCount`, `LostConstraintCount`, `OverallVerdict`, `BlockCount`, `ConcernCount`, `PassCount`, `TransitionSuccess`, `DiscriminatorConsulted`, `DiscriminatorActivated`, `DiscriminatorActivationReason`, `Scenarios[]`, `HumanResult`. The `DiscriminatorConsulted` and `DiscriminatorActivated` fields are the machine gates for the non-generalization proof.

Record: Metrics 17 fields with JSON snake_case names: `p3c_corrupted_packet_count`, `p3c_lost_constraint_count`, `p3c_overall_verdict`, `p3c_block_count`, `p3c_concern_count`, `p3c_pass_count`, `p3l_feedback_records_added`, `p3l_discriminators_added`, `p3l_over_generalization_risk`, `p3v_packet_count`, `p3v_lost_constraint_count`, `p3v_overall_verdict`, `p3v_block_count`, `p3v_concern_count`, `p3v_pass_count`, `p3v_discriminator_activated` (explicit JsonPropertyName), `experiment_success` (explicit JsonPropertyName).

The build pipeline inside BuildEvidence():

```
BuildPhaseContextPacket()          [shared with T34]
  → BuildCorruptedRun(context)
      → BuildCorruptedDomainPackets()   [9 packets, td06 absent]
      → BuildCorruptedScenarios()       [P3-C-1 BLOCK, P3-C-2 concern, P3-C-3 pass]
```

```
      → BuildCorruptedHumanResult()
  → BuildLearningCycle(corruptedRun)
      → FeedbackRecord P3-L-FB-001
      → LearnedDiscriminator P3-L-DISC-001
  → BuildVerificationRun(context, learningCycle)
      → BuildOriginalDomainPackets()    [10 packets, td06 present as concern]
      → BuildVerificationScenarios()    [P3-V-1 concern, P3-V-2 concern, P3-V-3 pass]
      → BuildVerificationHumanResult()
  → BuildMetrics(...)
  → BuildPassGates(...)
```

The key logic in `BuildVerificationRun()` is the discriminator activation check:

```
bool discriminatorActivated = lostConstraints > 0
    && learningCycle.LearnedDiscriminators
       .Any(d => d.PredicateName == "BaseService");
```

Because `lostConstraints = 0` in the verification run (td06 is present), `discriminatorActivated = false`. This is the machine proof that the discriminator was consulted but did not fire.

New file: `Tests/DynamicFlowP3Tests.cs`

Nine tests in class `DynamicFlowP3Tests`:

Test name	What it verifies
P3_Run_WritesJsonMarkdownAndMermaidEvidence	JSON, MD, MMD files created with matching run IDs
P3_CorruptedRun_ProducesBlockVerdict	OverallVerdict = BLOCK, 9 packets, TransitionSuccess = false, BlockCount = 1
P3_CorruptedRun_BlockAttributedToBaseServicePredicate	DroppedConstraint = BaseService, LostConstraintCount = 1, P3-C-1 scenario blocks
P3_CorruptedRun_CrossPhaseContextRemainsConcem	P3-C-2 verdict = concern, not block; P3-C-3 verdict = pass
P3_LearningCycle_StoresFeedbackRecord	FeedbackRecordsAdded = 1; SubExperimentId = P3-C; DroppedConstraint = BaseService
P3_LearningCycle_DerivesSpecificDiscriminator	DiscriminatorsAdded = 1; PredicateName = BaseService; Generalization = none
P3_VerificationRun_ProducesPassWithGaps	OverallVerdict = PASS_WITH_GAPS, 10 packets, BlockCount = 0, ConcernCount = 2
P3_VerificationRun_DiscriminatorConsultedButNotActivated	DiscriminatorConsulted = true, DiscriminatorActivated = false; human result confirms specificity
P3_PassGates_AllPass	All 8 gates pass; `experiment_success` = true in JSON; `p3v_discriminator_activated` = false in JSON

3.4 P3 Results

P3-C (corrupted run): Verdict: BLOCK, LostConstraintCount = 1, DroppedConstraint = BaseService, CorruptedPacketCount = 9, BlockCount = 1, ConcernCount = 1, PassCount = 1.

P3-L (learning cycle): FeedbackRecordsAdded = 1, DiscriminatorsAdded = 1, OverGeneralizationRisk = none. Discriminator P3-L-DISC-001: BaseService absent → BLOCK; BaseService as concern → PASS_WITH_GAPS. Specificity: BaseService only.

P3-V (verification run): Verdict: PASS_WITH_GAPS, PacketCount = 10, LostConstraintCount = 0, DiscriminatorConsulted = true, DiscriminatorActivated = false, ConcernCount = 2, BlockCount = 0, PassCount = 1.

All 8 pass gates passed. Test baseline: 1005/1005 → 1014/1014 (+9 tests) **Commit:** `8e4134b` pushed to main

The result confirmed three behaviors:

1. The transition gate correctly produces BLOCK when a high-confidence upstream constraint has no semantic neighbor in the downstream packet set.
2. The learning cycle produces a specific discriminator, not a general rule.
3. The discriminator is correctly inactive when the predicate is present as a concern, confirming no over-generalization.

---

4. Architecture: Before and After Summary

4.1 What Existed Before T34 Round B

Before the T34 Round B corrections, the architecture was documented and implemented as follows:

- The nine-node pipeline was proven by the E53 experiment.
- The semantic compiler produced predicate vectors, but the diagrams did not clearly show it as the first step before the pipeline.
- The learning loop existed in pieces (feedback, DPO, LoRA, idle gate) but was not connected to phase transitions.
- There was no Phase Context Packet, no Transition Gate, no Context Packet Store, and no Phase Learning Gap tracker.
- The development phases were described as deterministic methodology schemas with fixed sub-phase lists, rather than as adaptive learned topologies over predicate vectors.
- The four SVG diagrams reflected a mixed mental model that sometimes treated nodes as processing words and did not clearly separate predicates, domain tags, and learned signatures.

4.2 What Changed After T34 Round B

The four SVG diagrams were redrawn to reflect the correct predicate-first model.

The `DEVELOPMENT-PHASES-GUIDE-EXTENSION-v1.md` was corrected and extended with three new parts:

- Part 3 defines the three representation layers that must never be mixed: the 22 logical predicates (fixed), domain tags (metadata), and learned phase signatures (statistical).
- Part 4 defines the phase registry as a learned topology with phase IDs, human labels, predicate signatures, sub-phase routers, transition edges, minimum continuity thresholds, and conformance tests.
- Part 5 defines four experiment requirements (P1 through P4) for proving the design.

The data layer diagram acquired three new entity types with concrete fields: `PhaseTopology` (methodology as versioned data), `ContextPacketStore` (versioned phase predicate summaries), and `TransitionEvidence` (gate verdict records). The `PhaseLearningGap` entity was added to connect phase-level failures to the learning cycle.

The continuous learning diagram acquired a fourth path — Phase-Level Learning — that feeds transition evidence and phase learning gaps into both the idle scheduled learning path and the gap-driven cell creation path.

4.3 Code Structure Before and After

Before T34: No C# code for phase transitions, context packets, or transition gates. The `35.ExperimentsV2` project contained experiments E01 through E50 and the infrastructure for experiment registration and evidence writing.

After T34 (phase 24): `T34PhaseTransitionProof.cs` added 12 record types and 14 methods. `DynamicFlowT34Tests.cs` added 8 tests. Test count: 997 → 1005.

After P3 (phase 25): `P3LearningFromFailureProof.cs` added 5 new record types (FeedbackRecord, LearnedDiscriminator, LearningCycleResult, CorruptedRunResult, VerificationRunResult) and 12 methods. It reuses the upstream PhaseContextPacket and DomainPacket shapes from T34. `DynamicFlowP3Tests.cs` added 9 tests. Test count: 1005 → 1014.

4.4 The Predicate Boundary That Both Experiments Hinge On

The central technical insight proven by T34 and P3 together is that the same constraint ("BaseService must be inherited") can be in three distinct predicate states, and each state produces a different verdict:

State	Evidence	LostConstraintCount	Verdict
Fully covered	td06 in packet set, CoverageStatus = covered, Gate B present	0	PASS
Present as concern	td06 in packet set, CoverageStatus = concern	0	PASS_WITH_GAPS
Entirely absent	No packet mentions BaseService	1	BLOCK

T34 proved the second state (present as concern → PASS_WITH_GAPS). P3-C proved the third state (entirely absent → BLOCK). P3-V proved that the discriminator learned from state three does not misclassify state two. The first state (fully covered → PASS) is the natural target for follow-up: adding a Gate B assertion for BaseService is the concrete next action named in the T34 human result.

---

5. What Comes Next

P4 — Full Lifecycle Phase Topology

P4 will prove that a topology of phases can orchestrate a lifecycle across multiple phase runs. The first target topology is Phase 1 (UI/UX Definition) → Phase 3 (Technical Design) → Phase 4 (Implementation).

Pass conditions for P4: - Phase 1 produces a carry-forward predicate summary. - Phase 3 consumes the Phase 1 summary and preserves design constraints. - Phase 3 produces a carry-forward predicate summary. - Phase 4 consumes the Phase 3 summary and preserves technical constraints. - Both transition verdicts are correct. - Gap packets are attributed to the phase that produced the gap.

Gate B Closure for BaseService

The T34 human result named one concrete next action: add a Phase 4 Gate B assertion for inherited service constraints. This is the first specific remediation for a concern that the transition gate flagged but did not block.

Training Pairs for the BLOCK Path

Once the P3 evidence is validated, gap-closure training pairs can be written for node-07 (arbiter selector) and node-08 (arbiter units) to teach the BLOCK verdict path. The `examination-domain-`

`pairs/` folder already has 60 pairs for the concern and pass paths; the BLOCK path requires separate pairs.

Appendix — Experiment Identifiers and File Locations

Item	Location
T34 Round A oracle documents	`orchestrator experiment/T34-PHASE-TRANSITION-ROUND-A-2026-05-04/`
T34 Round B manifest	`orchestrator experiment/ROUND-MANIFEST-2026-05-04-T34-PHASE-TRANSITION-ROUND-B.md`
T34 C# experiment	`35.ExperimentsV2/Experiments/T34PhaseTransitionProof.cs`
T34 tests	`Tests/DynamicFlowT34Tests.cs`
T34 JSON evidence	`experiments-v2/T34-PHASE-3-TO-4-TRANSITION-GATE/results/run-2026-05-04-*.json`
P3 Round A oracle documents	`orchestrator experiment/P3-LEARNING-FROM-FAILURE-ROUND-A-2026-05-04/`
P3 Round B manifest	`orchestrator experiment/P3-LEARNING-FROM-FAILURE-ROUND-A-2026-05-04/ROUND-MANIFEST-*`
P3 C# experiment	`35.ExperimentsV2/Experiments/P3LearningFromFailureProof.cs`
P3 tests	`Tests/DynamicFlowP3Tests.cs`
P3 JSON evidence	`experiments-v2/P3-LEARNING-FROM-TRANSITION-FAILURE/results/run-2026-05-04-*.json`
Architecture SVGs	`orchestrator experiment/dynamic_flow_runtime_uml.svg`, `dynamic_flow_data_layer_uml.svg`, `continuous_learning_integration.svg`, `missing_tools_for_context_completion.svg`
Phases guide extension	`orchestrator experiment/DEVELOPMENT-PHASES-GUIDE-EXTENSION-v1.md`
Git commits	T34: `e6622a8` / P3: `8e4134b` — both on branch `main`

End of report. Produced: 2026-05-04.